

01/09/2026

Recursion Pattern

Level -1 : Foundations

① What is Recursion?

* Recursion is a technique where a function calls itself to solve a smaller version of the same problem.

Example:

```
def count-down(n):
```

```
    if n == 0:
```

```
        return
```

```
    print(n)
```

```
    count-down(n-1)
```

```
count-down(3)
```

Count-down(3)

↓

(2)

↓

(1)

↓

(0)

↓

Stop

② Base case : Base case is the condition that stops the Recursion.

Example:

```
if n == 0:
```

```
    return
```

Here $n = 0$

↓

Stop

without a base case the recursion continues...

③ Recursive case : Recursive case is the part where the function calls itself with a smaller/simpler input.

Example: Count-down(n-1)

Here,

n

↓

n-1

↓

n-2

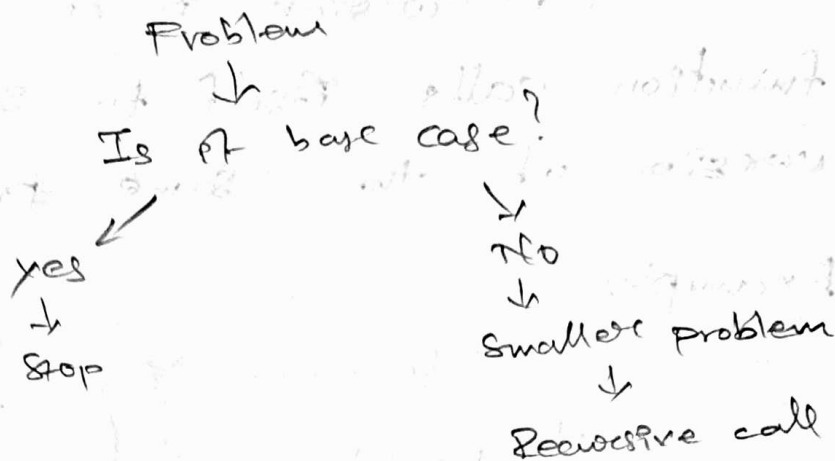
↓ ...

}

Problem is becoming smaller.

④ Basic Structure :

```
def solve(problem):
    if base_case:
        return base_value
    return solve(smaller_problem)
```



⑤ How do we make the problem smaller?

factorial(5) → Instead of solving $5!$ directly

we use; $5! = 5 \times 4 \times 3 \times 2 \times 1$

$$\begin{aligned}
 5! &= 5 \times 4! \\
 4! &= 4 \times 3! \\
 3! &= 3 \times 2! \\
 2! &= 2 \times 1!
 \end{aligned}
 \left. \vphantom{\begin{aligned} 5! &= 5 \times 4! \\ 4! &= 4 \times 3! \\ 3! &= 3 \times 2! \\ 2! &= 2 \times 1! \end{aligned}} \right\} \begin{aligned} &\text{So,} \\ &\text{factorial}(n) \\ &\downarrow \\ &n \times \text{factorial}(n-1) \end{aligned}$$

⑥ Recursion vs Iterations :

Iteration :

```
for i in range(3, 0, -1):
    print(i)
```

It produces :

3
2
1 } o/p

Recursion :

```
def func(n):
    if n == 0:
        return
```

print(n)

func(n-1)